3.6 PPT

3.6 LangGraph 与 Agent 编排 v2

本节要解决的问题

开发者提交一张上线工单，系统如何自动跑完从构建、单测到发布上线，并支持人工审批。

发布流程图如下

代码块

```
START → 受理申请 → [并行 ToolNode ×4: 构建/单测/安全扫描/依赖审计]
        → 汇合 → 变更等级路由(L/M/H)
          ├─ L → 自动放行
          ├─ M → Tech Lead 审批 (interrupt)
          └─ H → Tech Lead + Security 双审批 (interrupt ×2)
        → 预发部署 → 冒烟测试 ──失败──┐
                          │成功        │
                          ↓            ↓
                      灰度 10% → SRE 确认 (interrupt) → 监控
                          │异常 ↘                    ↗ 正常
                          │      自动回滚循环 (≤3 次)
                          ↓
                      灰度 50% → 100% → 完成报告 → END
```

这张图覆盖了 LangGraph 的 6 个核心机制：

- **图 / 节点 / 边 / 状态**：每个方框是节点，方框间连线是边，工单本身在节点间流动的数据是状态
- **StateGraph 条件分支与循环**：变更等级路由、冒烟失败回退、灰度异常回滚，3 个分支 2 个循环
- **ToolNode**：四路并行检查的执行体
- **Interrupt**：M/H 审批和每次灰度推进的人工确认
- **Checkpoint**：H 级审批可能跨天，进程也得能停得下来 + 重启接得上
- **调试与可视化**：Mermaid（一种"用文字描述流程图、工具自动画成图"的格式）、`astream` 流式输出、`langgraph dev` 本地 HTTP 服务

一、区分概念

名称	解决什么问题	不解决什么	本课用法
LangGraph	用"状态 + 节点 + 边"把流程编排成可执行的图，调度分支/循环/中断/恢复	不直接帮你写 Prompt，不做模型抽象	**主角**，本章全部能力都靠它
LangChain	模型客户端 + 消息类型 + 工具接口的统一封装	不管你怎么调度多步骤流程	配角：用它的 `@tool` 和消息类型
AgentScope	另一套独立的 Agent 框架，模型/工具/协作都自己一套	与 LangGraph 不是插件关系	知道即可，不装
Dify	可视化搭建 Workflow 的平台，拖拽节点	不适合做"代码内编排 + 跨进程恢复"这种工程化要求高的事	知道即可，不部署
LangSmith / Langfuse	观测平台，记录每一次调用的 Trace（输入、输出、耗时）	不参与执行；不会让任务具备"中断后继续"的能力	可选接入，本章不依赖

本章只用 `langchain-openai` / `langchain-deepseek` 做模型客户端、`langchain-core` 负责消息类型、其余都靠 LangGraph 本体。

二、业务拆解

核心难点

- 不是 API 怎么调，而是业务步骤拆得对不对
- 先明确：串行 / 并行 / 分叉 / 停下等人

拆解结果：7 个业务步骤

1. 受理申请：校验工单字段，准备上下文

2. 前置检查：构建、单测、安全扫描、依赖审计（可并行）
3. 变更分级：按 `patch_diff` 判 L / M / H
    - 判据：敏感词（auth、payment）+ 改动长度
    - 等级越高，审批越严
4. 审批：按等级决定是否人工介入、几人介入
5. 预发部署 + 冒烟：部署 staging，跑冒烟；失败→打回修订，重跑
6. 灰度推进：10% → 50% → 100%；每段 SRE 确认 + 健康监控
7. 完成报告：汇总审批与阶段结果；工单标记最终状态（成功 / 被拒 / 回滚）

一句话

动手前先拆对流程，再写 LangGraph 代码。

把 7 步业务映射到 LangGraph 的"对象"：

业务概念	LangGraph 对象
工单本身（贯穿整个流程的共享数据）	**State**：一个随流程一路传递的字典，每个节点从它读数据、把结果写回它
每个步骤	**节点**（一个 Python 函数）
步骤之间的依赖关系	**边**（`add_edge` / `add_conditional_edges`）
流程总装	**图**（`StateGraph`）
等待人工输入	**Interrupt**（`interrupt()` + `Command(resume=...)`）
跨进程暂停与恢复	**Checkpoint**（`SqliteSaver` / 顶层工程用 `AsyncSqliteSaver`）
节点要调用的外部能力	**ToolNode**（一个"拿着工具清单、负责并发执行工具"的现成节点）

关键认知：节点 ≠ 一次模型调用

- 节点 = 业务步骤
- 内部实现由节点决定：纯函数 / 调模型 / 跑工具

三、主线

目标：先跑通最直主线

受理 → 4 项前置检查（全通过）→ 预发 → 冒烟 → 灰度 10% → 50% → 100% → 报告

主线特征

- 无分支
- 无中断
- 无人工审批
- 全程无失败

策略

先跑通最直路径，后续逐步加机制。

三步走

1. AI 生成代码：按提示词生成
2. 跑测试验证
3. 回读生成内容

验收标准（AC）

- 写成可执行 pytest 测试用例
- 以测试全部通过为准
- 提示词按 P1…P6 编号，每节一份

3.1 提示词 P1

代码块

```
任务：用 LangGraph 生成一个微服务发布审批流水线工程的"主线最小版"。

【工程目录】release_pipeline/demos/01_minimal/
【要生成的文件】
  state.py     状态 TypedDict
  nodes.py     7 个节点函数
  graph.py     StateGraph 装配
  app.py       一行命令能跑完的 CLI
  tests/test_minimal.py  验收测试

【业务目标（本步骤只跑主线，不含任何分支）】
开发者提交工单 → 跑 4 项前置检查（构建/单测/安全扫描/依赖审计，全部直接通过）
→ 预发部署 → 冒烟测试 → 灰度 10% → 灰度 50% → 灰度 100% → 输出发布报告。

【State 字段（TypedDict，使用下列字段，不要新增）】
```

```
class ReleaseState(TypedDict):
    ticket_id: str
    service: str
    version: str
    checks_passed: int
    stages_completed: NotRequired[list[str]]
    outcome: NotRequired[str]
    report_path: NotRequired[str]

【节点清单（7 个，函数命名锁定）】
intake          受理 + 跑 4 项检查（用同步内联函数模拟，不要调工具）
deploy_staging  往 stages_completed 追加 "staging"
smoke_test      追加 "smoke"
canary_10       追加 "canary_10"
canary_50       追加 "canary_50"
canary_100      追加 "canary_100"
finalize        写 reports/{ticket_id}.json，置 outcome="released"

【边（线性，普通 add_edge，不要条件路由）】
START → intake → deploy_staging → smoke_test → canary_10 → canary_50 →
canary_100 → finalize → END

【Checkpointer】用 langgraph.checkpoint.memory.InMemorySaver()，不要 SQLite。

【验收 AC（写成 pytest 用例，每条一个 test_ 函数）】
AC-1: ainvoke 跑完后 outcome == "released"
AC-2: stages_completed 依次为
["staging","smoke","canary_10","canary_50","canary_100"]
AC-3: checks_passed == 4
AC-4: report_path 文件存在且能加载

【明确不做】
- 不引入 langchain 工具、不引入 @tool、不引入 ToolNode
- 不做 interrupt、不做 Command
- 不做条件路由（add_conditional_edges 一次都不要出现）
- 不做循环（任何回边都不要）
- 不写多租户、权限、Web UI

【风格】节点是函数不要类；状态用 TypedDict；类型注解齐全；不写注释；
所有文件总行数控制在 200 行以内。

【依赖】langgraph>=0.2 / Python 3.11+ / pytest。无需 DeepSeek 密钥。
```

3.2 跑起来验证

代码块

```
cd release_pipeline/demos/01_minimal
python app.py
```

实际输出（节选，`...` 表示截断的长列表；截图前请清掉 `reports/`）：

代码块

```
▶ intake: {'checks_passed': 4, 'stages_completed': []}
▶ deploy_staging: {'stages_completed': ['staging']}
▶ smoke_test: {'stages_completed': ['staging', 'smoke']}
▶ canary_10: {'stages_completed': ['staging', 'smoke', 'canary_10']}
▶ canary_50: {'stages_completed': [..., 'canary_50']}
▶ canary_100: {'stages_completed': [..., 'canary_100']}
▶ finalize: {'outcome': 'released', 'report_path': .../reports/DEMO-01.json'}
```

节点按边顺序推进，State 增量可见，主线发布成功。

- 运行方式：`app.py` 以 `stream_mode="updates"` 逐步推送
- 每行格式：`▶ 节点名：本步 State 更新`
- 首行 `intake`：`checks_passed=4`，4 项检查全通过；`stages_completed=[]`
- 中间行：`stages_completed` 逐项追加 `staging → smoke → canary_10 → canary_50 → canary_100`，与边顺序一致
- 末行 `finalize`：`outcome="released"`，报告写入 `reports/DEMO-01.json`
- 测试结论：`4 passed` 全部通过

3.3 三个关键概念

1. State：用 TypedDict 声明

`demos/01_minimal/state.py`

代码块

```
class ReleaseState(TypedDict):
    ticket_id: str
    ...
    stages_completed: NotRequired[list[str]]
```

- 声明字段与类型，便于检查拼写/类型

- 运行时不拦截额外字段
- `NotRequired`：初始可缺省，后续节点补

2. 节点：只返回要更新的字段

`nodes.py`

```
代码块
1  def deploy_staging(state: ReleaseState) -> dict:
2      return {"stages_completed": state["stages_completed"] + ["staging"]}
```

- 接收整个 State，但只返回增量
- 不能返回整个 State：
  - 并行节点会同时更新同字段
  - 无 reducer → 并发更新错误
  - 有 reducer → 可能重复合并
- 与普通 pipeline 不同：
  - pipeline：全量对象传递
  - LangGraph：可并行、可重跑、每步存档 → 节点报增量，框架统一合并

3. 图：构建器 + compile

`graph.py`

```
代码块
1  builder = StateGraph(ReleaseState)
2  builder.add_node("intake", nodes.intake)
3  ...
4  builder.add_edge(START, "intake")
5  builder.add_edge("intake", "deploy_staging")
6  ...
7  graph = builder.compile(checkpointer=InMemorySaver())
```

- `StateGraph` 是构建器，`compile()` 才得到可调用对象
- `compile()` 只做结构检查，不查业务规则
- 必须传 checkpointer：`InMemorySaver`

- 每步存 State 快照，为后续 `interrupt` 恢复做准备
- 没有 checkpointer， `Command(resume=...)` 会报错

主线价值：得到一张能跑通的图。后续每加一个机制，都是往这张图上叠加。

下一节：第一个机制——条件路由。

四、分支：条件路由

为什么加分支

- “改一行 README” ≠ “改 auth 模块”
- 不同风险，必须走不同审批路径

本步做法

- `intake` 后加一步：变更分级
- 依据： `patch_diff` （工单中的代码改动内容）
- 判定：L / M / H 三级
- 按等级路由到不同审批节点

4.1 提示词 P2

代码块

```
任务：在 demos/01_minimal/ 主线最小版的基础上，
新建 demos/02_routing/，加一个"变更等级路由"分支。

【本步骤业务目标】
工单受理后多走一步"变更分级"：根据 patch_diff 内容把变更打成 L / M / H 三档，
L 走自动放行，M 走 Tech Lead 一档审批，H 走双审批。
本步骤不引入 interrupt，审批节点先用占位函数模拟通过。

【State 在 01 基础上新增字段】
patch_diff: str
change_level: NotRequired[Literal["L","M","H"]]
approver_count: NotRequired[int]

【节点清单（在 01 的 7 个节点基础上新增 4 个）】
```

```
classify_change     根据 patch_diff 计算等级，规则：
                    含 auth/payment/password/token → "H"
                    len(diff) > 800 → "M"
                    其余 → "L"
approval_auto_L     approver_count=0
approval_lead_M     approver_count=1
approval_dual_H     approver_count=2

【边（关键变化）】
START → intake → classify_change
classify_change 接 add_conditional_edges，路由函数：
  def route_by_level(state) -> Literal["L","M","H"]: return
state["change_level"]
映射：{"L":"approval_auto_L","M":"approval_lead_M","H":"approval_dual_H"}
三个审批节点都各自 add_edge → deploy_staging
之后接 01 的 smoke_test → canary_10 → canary_50 → canary_100 → finalize →
END

【Checkpointer】沿用 InMemorySaver()。

【验收 AC】
AC-1: 短 diff → change_level=="L", approver_count==0, outcome=="released"
AC-2: 长度 1200 的 diff → change_level=="M", approver_count==1
AC-3: 含 "auth token" 的 diff → change_level=="H", approver_count==2
AC-4: 三种 diff 都必须把 stages_completed 完整走完

【明确不做】
- 不引入 @tool、ToolNode（下一步）
- 不引入 interrupt、Command（更后面的步骤）
- 不引入循环、不引入 SQLite Checkpointer、不接 DeepSeek
```

4.2 跑起来验证

代码块

```
cd demos/02_routing
python app.py         # 三种 diff 走三条不同分支
python -m pytest tests/ -v
```

输出演示：

```
R-02-L: ▶ classify_change: {'change_level': 'L'} → ▶ approval_auto_L: {'approver_count': 0}
R-02-M: ▶ classify_change: {'change_level': 'M'} → ▶ approval_lead_M: {'approver_count': 1}
R-02-H: ▶ classify_change: {'change_level': 'H'} → ▶ approval_dual_H: {'approver_count': 2}
→ 4 passed in 0.21s
```

工单	patch_diff	判定	审批节点	审批人数
R-02-L	短配置改动	L	approval_auto_L	0
R-02-M	1200 字符长改动	M	approval_lead_M	1
R-02-H	含 auth token	H	approval_dual_H	2

`classify_change` 判级，路由分流，L / M / H 各走各的审批。

4.3 关键概念

`02_routing/graph.py`：

代码块

```python
def route_by_level(state: ReleaseState) -> Literal["L", "M", "H"]:
    return state["change_level"]

builder.add_conditional_edges("classify_change", route_by_level, {
    "L": "approval_auto_L",
    "M": "approval_lead_M",
    "H": "approval_dual_H",
})
```

业务需求：风险分级审批

- L：自动放行
- M：Tech Lead 审批
- H：Tech Lead + Security 双审批
- `route_by_level`：读 `change_level`，返回 `L / M / H`

1. 条件路由函数

`add_conditional_edges(source, path_fn, path_map)`

- `source`：分叉点，节点执行完后开始选择
- `path_fn`：选路函数，只读 State，返回路名
- `path_map`：路名 → 节点名
- 路由函数不改状态；要改 State，就专门加节点

2. 返回类型用 `Literal[...]`

- 不是装饰，是图元信息
- 给 `compile()` 和 Mermaid 用
- 用 `str` 能编译，但分支标签会丢

3. 分支末端汇合

- 每个分支各自 `add_edge(...) → deploy_staging`
- 多条边指向同一目标 = 汇合
- 与下一节 ToolNode 并行不同：
  - ToolNode 是节点内部并发
  - 图上只是一个普通节点

五、并行：ToolNode

问题

- 4 项检查互不依赖，各自耗时
- 构建 / 单测 / 安全扫描 / 依赖审计
- 串行执行 = 4 倍耗时

需求

- 互不依赖的子任务同时开工

方案

- LangGraph 现成执行器：`ToolNode`
- 你提供工具清单，它负责并发执行

5.1 提示词 P3

代码块

```
任务：在 demos/02_routing/ 的基础上，新建 demos/03_toolnode/，
把"4 项前置检查"从内联函数升级为 LangGraph 的 ToolNode 并行调度。

【本步骤业务目标】
构建、单测、安全扫描、依赖审计这 4 项检查互不依赖，要让它们在图上"并行执行"。

【State 在 02 基础上新增字段】
messages: NotRequired[Annotated[list[AnyMessage], add_messages]]
checks:   NotRequired[list[CheckResult]]

【新增文件 tools.py：4 个 @tool 函数（mock，返回 JSON 字符串）】
build_service(service, version)
run_unit_tests(service)
security_scan(diff)
dependency_audit(service)

【节点变化】
intake 不再内联跑检查，改为返回一条 AIMessage，里面带 4 个 tool_calls
新增节点：tool_checks = ToolNode(CHECK_TOOLS)
新增节点：collect_checks，解析 ToolMessage → checks 列表 + checks_passed

【边变化】
原：START → intake → classify_change → ...
新：START → intake → tool_checks → collect_checks → classify_change → ...

【验收 AC】
AC-1: state["checks"] 长度为 4，name 齐全
AC-2: state["checks_passed"] == 4
AC-3: 路由分支与 02 一致
AC-4: state["messages"] 末尾至少 4 条 ToolMessage

【明确不做】
- 不引入 interrupt、Command、SQLite Checkpointer、循环
- 4 个工具就是 4 个 mock 函数，不调真实大模型
```

5.2 跑起来验证

代码块

```
cd demos/03_toolnode
python app.py          # 11 行节点事件，4 项检查全过
```

```
python -m pytest tests/ -v
```

输出结果：

代码块

```
▶ intake: ['messages', 'stages_completed']
▶ tool_checks: ['messages']                ← ToolNode 内部并行了 4 个工具
▶ collect_checks: ['checks', 'checks_passed']
▶ classify_change: ['change_level']
▶ approval_dual_H: ['approver_count']      ← 路由选 H 分支
▶ deploy_staging / smoke_test / canary_* / finalize
→ level=H checks_passed=4
→ 4 项检查: ['build', 'unit_test', 'security_scan', 'dep_audit']
```

- 只打印字段名，不打印值（消息列表太长）
- `intake`：写入 `messages`，含“调用 4 个工具”指令
- `tool_checks`：即 ToolNode，只更新 `messages`
  - 4 个工具在一个节点内部并发完成
  - 图上只占一行，看不出 4 条线
- `collect_checks`：读 4 条 ToolMessage → `checks` + `checks_passed`
- 终态：等级 H，4 项检查全过（构建 / 单测 / 安全扫描 / 依赖审计）

5.3 四个关键概念

`03_toolnode/nodes.py::intake`：

代码块

```
def intake(state: ReleaseState) -> dict:
    fanout = AIMessage(
        content="",
        tool_calls=[
            {"name": "build_service", "args": {...}, "id": "call_build"},
            {"name": "run_unit_tests", "args": {...}, "id": "call_unit"},
            {"name": "security_scan", "args": {...}, "id": "call_scan"},
            {"name": "dependency_audit", "args": {...}, "id": "call_audit"},
        ],
    )
    return {"messages": [fanout], "stages_completed": []}
```

1. 并行从哪来

- ToolNode 不看图，只看 `messages` 末尾的 `AIMessage`
- 一条 `AIMessage` 携带 4 个 `tool_calls` → 自动并发执行 4 个工具
- 结果包成 `ToolMessage` 追加到 `messages`
- 这就是 ToolNode 的契约

`03_toolnode/graph.py`：

代码块

```
from langgraph.prebuilt import ToolNode
from tools import CHECK_TOOLS

builder.add_node("tool_checks", ToolNode(CHECK_TOOLS))
builder.add_edge("intake", "tool_checks")
builder.add_edge("tool_checks", "collect_checks")
```

2. ToolNode 是什么

- 预实现好的节点对象，不用写函数
- 注册后自动：
  ① 取最后 `AIMessage`
  ② 按 `tool_calls` 找工具
  ③ 并发执行
  ④ 结果包 `ToolMessage` 写回 `messages`

`03_toolnode/nodes.py::collect_checks`：

代码块

```
def collect_checks(state: ReleaseState) -> dict:
    results: list[CheckResult] = []
    for msg in state.get("messages", []):
        if not isinstance(msg, ToolMessage):
            continue
        payload = json.loads(msg.content)
        results.append(CheckResult(...))
    return {"checks": results, "checks_passed": sum(1 for r in results if r["passed"])}
```

3. messages 的 reducer

- 必须声明：`Annotated[list[AnyMessage], add_messages]`
- reducer：多个节点写同一字段时，决定新旧值怎么合并
- `add_messages`：追加 + 按消息 id 去重
- 不加 reducer：`intake` 的 `AIMessage` 会被后续节点覆盖

4. 写 messages：加一条，不是换一批

- `{"messages": [fanout]}` → 追加到列表末尾
- 再发 `tool_calls` 注意：用新 id，或显式清空（`RemoveMessage`）
- `collect_checks` 保留全部消息：审计需要

当前进度
主线 + 分支 + 工具并行已就位；审批仍是占位。
下一节：`interrupt` 暂停等人批。

六、中断与恢复：interrupt + Checkpoint

增加三个主要逻辑

1. 审批节点改真 `interrupt()`
2. `Command(resume=...)` 送回人工决定
3. `InMemorySaver` → SQLite Checkpointer，支持跨进程暂停 / 退出 / 重启 / 恢复

代码块

```
M 级:   approval_m ──(停在 INT-1，等 Tech Lead)──→ 批准 → 预发部署 …
                                                  └→ 拒绝 → finalize（结束）

H 级:   approval_h_lead ──(停在 INT-2A，等 Tech Lead)──→ 批准 ─→
         approval_h_security ──(停在 INT-2B，等安全负责人)──→ 批准 → 预发部署 …
              任一处拒绝 → finalize（结束）
```

6.1 提示词 P4

```
任务：在 demos/03_toolnode/ 的基础上，新建 demos/04_hitl/，
把"3 个审批节点的占位返回值"升级为真正的 interrupt 暂停 + Command 恢复 +
SQLite Checkpoint 跨进程持久化。

【本步骤业务目标】
M 级需 tech_lead 批准（INT-1）；H 级需两段串行：先 tech_lead（INT-2A）再 security
（INT-2B）。
图执行到审批节点要暂停并返回待批内容，用户稍后提交决定再继续；
进程退出后凭同一个 thread_id + 同一个 SQLite 文件能找回待批位置。

【State 在 03 基础上新增字段】
class Approval(TypedDict):
    interrupt_id: Literal["INT-1","INT-2A","INT-2B"]
    role:   Literal["tech_lead","security"]
    action: Literal["approve","reject"]
    actor: str
    feedback: str
approvals: NotRequired[Annotated[list[Approval], add]]   # add reducer：审批累加
审计

【新增 checkpointer.py】
用同步 SqliteSaver（langgraph.checkpoint.sqlite）。
@contextmanager open_checkpointer()：读 env CHECKPOINT_DB（默认
./checkpoints_04.sqlite），
mkdir 父目录后 from_conn_string 打开，yield saver。

【nodes.py 三个审批节点改成真 interrupt】
approval_m / approval_h_lead / approval_h_security 各自 decision =
interrupt({...})，
用 _as_approval(decision, iid, role) 把 resume 回来的 dict 转成 Approval，
返回 {"approvals": [...]}（只放本次新增的一条）。approval_auto_L 仍不 interrupt。

【graph.py 边变化】
三个审批节点后全改成 add_conditional_edges：
  route_after_approval(state)：任一 approval.action=="reject" → "reject"，否则
"approve"
  approval_m        → approve:deploy_staging / reject:finalize
  approval_h_lead   → approve:approval_h_security / reject:finalize
  approval_h_security → approve:deploy_staging / reject:finalize

【新增 app.py：submit / status / approve 三个命令行】
待审批从 snapshot.tasks[*].interrupts[*].value 读取，不要用 __interrupt__ 旧写法。

【验收 AC】
AC-1：M 级（长 diff）invoke 后 snap.next==("approval_m",)，
interrupts[0].value["interrupt_id"]=="INT-1"
AC-2：resume approve 后 outcome=="released"，approvals 角色==["tech_lead"]
```

```
AC-3: H 级先停 INT-2A，resume 后停 INT-2B，再 resume 后 released，approvals 长度==2
AC-4: 跨进程——第一个 context 跑到 INT-1 暂停后关闭；新进程 open_checkpointer() 打开同一个 SQLite，get_state 找回 ticket/next，Command(resume) 后 released
AC-5: reject 短路：M 级 resume reject 后 outcome=="rejected"，stages_completed==[]

【明确不做】
- 不引入循环/重试/回滚（下一步）；不引入可视化、langgraph dev（下一步）
- 审批决定全部由 Command(resume=...) 人工传入，不调真实大模型
- 不用 AsyncSqliteSaver，本格用同步 SqliteSaver 让 CLI 演示更直接
```

6.2 跑起来验证

代码块

```
cd demos/04_hitl
python -m pytest tests/ -v
```

执行结果：

代码块

```
tests/test_hitl.py::test_ac1_pause_at_INT_1 PASSED
tests/test_hitl.py::test_ac2_resume_finishes_M PASSED
tests/test_hitl.py::test_ac3_H_two_sequential_interrupts PASSED
tests/test_hitl.py::test_ac4_cross_process_resume PASSED
tests/test_hitl.py::test_ac5_reject_short_circuits_to_finalize PASSED
5 passed
```

手动测试：

代码块

```
python app.py submit T-042 auth-svc 1.0.0 "$(python -c 'print("x"*1200)')"
python app.py status T-042
python app.py approve T-042 approve alice "看起来没问题"
python app.py status T-042
```

执行结果：

```
代码块ext=('approval_m',) outcome=None
⏸️  待审批:
    INT-1 role=tech_lead ticket=T-042
after resume: next=() outcome=released
"stages_completed": ["staging","smoke","canary_10","canary_50","canary_100"]
```

阶段	输出	含义
submit	T-042：auth-svc / 1.0.0 / 1200 字符 diff	长度 >800 → 判 M 级
暂停	next=('approval_m',) outcome=None	停在 M 级审批；未往下走；进程已退出
status	INT-1 role=tech_lead ticket=T-042	从 SQLite 读回待批内容：T-042 等 Tecl
approve	next=() outcome=released	批准后恢复运行并跑完；发布成功
再 status	staging, smoke, canary_10, canary_50, canary_100	预发、冒烟、三档灰度全部完成

6.3 关键代码

`04_hitl/nodes.py::approval_m`：

代码块

```python
def approval_m(state: ReleaseState) -> dict:
    decision = interrupt({
        "interrupt_id": "INT-1",
        "role": "tech_lead",
        "ticket_id": state["ticket_id"],
        ...                          # 载荷里还带了 service / version，此处省略
        "diff_preview": state.get("patch_diff", "")[:300],
        "actions": ["approve", "reject"],
    })
    return {"approver_count": 1,
            "approvals": [_as_approval(decision, "INT-1", "tech_lead")]}
```

1. interrupt 如何“停”

- `interrupt(payload)` 抛出特殊信号，图停下
- `payload`：给审批人看的可 JSON 数据
- 首次执行：不会走到下面 `return`
- 恢复执行：`interrupt()` 返回 `Command(resume=...)` 的值 → `return` 执行 → 写回 State

2. 最大的坑：恢复会从头重跑节点

- LangGraph 不保存 Python 调用栈，只保存 State
- 恢复时整个节点函数重新执行
- 铁律：`interrupt()` 之前禁止副作用（写文件、发请求、扣款、应用补丁）
- 正确布局：
  - `patch_diff`：submit 入口写入
  - 前序节点：只读、校验、分级
  - 审批节点：只展示 + 收决定
  - 部署 / 切流 / 写报告：放在审批通过后

`04_hitl/checkpointer.py`：

代码块

```
@contextmanager
def open_checkpointer():
    db_path = Path(os.environ.get("CHECKPOINT_DB", "./checkpoints_04.sqlite")).resolve()
    db_path.parent.mkdir(parents=True, exist_ok=True)
    with SqliteSaver.from_conn_string(str(db_path)) as saver:
        yield saver
```

3. interrupt 生效条件

- `compile()` 必须传 checkpointer
- 调用必须带 `thread_id`
- `thread_id` ≠ 操作系统线程
  - 是“一次独立任务”的编号，相当于存档名
  - 同文件 + 同 `thread_id` → 跨进程 `get_state()` 找回
- `thread_id` 只是查找键，不是权限；审批权限仍需入口校验

`04_hitl/app.py::approve`：

代码块

```
snap = graph.get_state(cfg)
if not snap.next:
    print("该工单没在等待审批。")
```

```
        return
    graph.invoke(
        Command(resume={"action": action, "actor": actor, "feedback": feedback}),
        config=cfg,
    )
```

4. resume + reducer

- 恢复用 `Command(resume=...)`，不是重提交任务
- 必须由“收到人工决定”的入口触发，不能 submit 后自动批准
- `approvals` 用 `add` reducer：
  - 每个审批节点只返回自己这条
  - LangGraph 累加 → 完整审计轨迹
  - `route_after_approval` 可 `any(reject)` → 短路到 `finalize`
  - reject 后 `stages_completed == []`

七、循环与预算：回边重跑 + 计数兜底

循环 A：修订

- 审批新增 `revise`
- `revise` → `begin_revision`：`revision_count + 1` → 普通边退回 `intake`
- 重跑整条链路：检查 → 分级 → 审批
- 触顶：路由判 `stop` → 终止，记 `incomplete`

循环 B：灰度回滚

- 每档灰度部署后拉指标
- 不健康 → `auto_rollback`：`rollback_count + 1`，游标回退
- 条件边退回 `canary_step`，重跑同一阶段
- 触顶：`abort` → 终止，记 `rolled_back`

循环让流程可重跑；预算计数防死循环。

7.1 提示词 P5（大部分由 AI 生成）

```
任务：在 demos/04_hitl/ 的基础上，新建 demos/05_loops/，加两处"循环"，
并给每处循环配一个预算上限，保证不会无限打转。

【State 在 04 基础上新增字段】
revision_count / max_revisions / rollback_count / max_rollbacks / canary_cursor
（入口给值）
current_stage: NotRequired[dict]
metric_samples: NotRequired[Annotated[list[MetricSample], add]]
Approval.action 增 "revise"、"abort"; outcome 增 "rolled_back"、"incomplete"

【新增工具】deploy(stage,service,version) / query_metrics(service,stage)（mock，
返回 JSON）

【nodes.py 新增/改动】
begin_revision: {revision_count+1, checks:[]}
canary_step: cursor=canary_cursor+1; decision=interrupt(INT-3-<stage>,role=sre,
[approve,abort]);
             abort→{outcome:"rejected"}; 否则
{canary_cursor:cursor,current_stage}
monitor_window: flaky 服务 canary_10 首遍判不健康，该阶段回滚过即判健康;
                metric_samples 追加 + current_stage.status=healthy/unhealthy
auto_rollback: {rollback_count+1, canary_cursor:max(0,cursor-1),
                stages_completed+[f"rolled_back:{stage}"]}

【graph.py：循环靠"条件边指回更早的节点"】
route_approval 读 approvals[-1].action:
  approve→deploy_staging / reject,abort→finalize /
  revise→(revision_count>=max_revisions? stop→finalize : begin_revision)
begin_revision → add_edge 回 intake                    # 循环 A
deploy_staging → canary_step → route_after_canary → monitor_window
monitor_window → route_after_monitor {advance→canary_step,
unhealthy→auto_rollback, released→finalize}
auto_rollback  → route_after_rollback {retry→canary_step, abort→finalize}
# 循环 B

【验收 AC】
AC-1: revise 后 revision_count==1 且仍停在 approval_m
AC-2: revise→approve→3×SRE 后 released，revision_count==1
AC-3: max_revisions=1 连点两次 revise → revision_count==1、next==()、
outcome==incomplete
AC-4: flaky+max_rollbacks=2，approve+4×SRE 后 rollback_count==1、released、含
rolled_back: 项
AC-5: flaky+max_rollbacks=0 → rollback_count==1、outcome==rolled_back、next==()

【明确不做】不引入可视化（下一步）；不接真实模型/metrics；approvals、metric_samples
用 add reducer
```

7.2 跑起来验证

代码块

```
cd demos/05_loops
python -m pytest tests/ -v    # 5 条 AC
python app.py                 # 两个 demo 场景，看循环真正生效
```

测试结果：

代码块

```
AC-1..AC-5  5 passed

== 场景 A：修订循环（先 revise 再 approve）==   revision_count=1 outcome=released
== 场景 B：灰度回滚后重试（flaky 服务）==       rollback_count=1  outcome=released
```

- `5 passed`：P5 五条 AC 全部通过，含“修订上限耗尽”“回滚预算为零”等边界。
- 场景 A：先 `revise` → 回到 `intake` 重跑；`revision_count=1`，最终 `released`。
- 场景 B：flaky 服务首次进入 `canary_10` 判不健康 → 回滚一次；`rollback_count=1`，最终 `released`。
- 说明：`canary_10` 是 10% 流量档位；`flaky` 是演示服务名，用于稳定触发回滚路径。

7.3 四个关键点

`05_loops/graph.py`：

代码块

```
b.add_edge("begin_revision", "intake")          # 循环 A：普通边指回更早的节点

b.add_conditional_edges("monitor_window", route_after_monitor, {
    "advance": "canary_step",                   # 同阶段健康 → 推进
    "unhealthy": "auto_rollback",               # 不健康 → 回滚
    "released": "finalize",
})
b.add_conditional_edges("auto_rollback", route_after_rollback, {
    "retry": "canary_step",                     # 循环 B：回滚后重跑同一阶段
    "abort": "finalize",
})
```

1. 循环无需专用 API

- `add_edge("begin_revision", "intake")` 指回已走过节点
- 节点写状态，路由读状态
- 每圈计数递增 → 路由适时不再选回边

2. 无预算 = 死循环

代码块

```
def route_approval(state: ReleaseState) ->
Literal["approve","revise","reject","stop"]:
    last = state.get("approvals", [{}])[-1].get("action")
    if last == "approve": return "approve"
    if last in ("reject", "abort"): return "reject"
    if state["revision_count"] >= state["max_revisions"]:
        return "stop"           # 预算耗尽 → finalize，不再 revise
    return "revise"
```

- 每循环配计数 + `>= max` 兜底
- 示例：`route_approval` 判 `revision_count >= max_revisions` → `stop`
- `recursion_limit` 是安全阀，抛 `GraphRecursionError`，非业务收尾
- 业务上限必须自己写成路由

`05_loops/nodes.py::canary_step`：

代码块

```
def canary_step(state: ReleaseState) -> dict:
    cursor = state["canary_cursor"] + 1        # ← 先算，但不提交
    stage = STAGE_BY_CURSOR[cursor]
    decision = interrupt({"interrupt_id": f"INT-3-{stage}", "role": "sre",
...})
    ...
    return {"canary_cursor": cursor, ...}      # ← 只有 return 时 cursor 才写进
State
```

3. interrupt 在循环里的坑

- 恢复时节点从头重跑
- `cursor = canary_cursor + 1` 是局部变量，`return` 才写 State → 不会重复 +1
- 前提：`interrupt()` 前无副作用

- `deploy` 放在审批通过后

4. 回滚如何原地重试

- `canary_cursor`：1=10%，2=50%，3=100%
- `auto_rollback` 游标 -1 → 下圈 `canary_step` +1 → 同一 stage
- `stages_completed` 追加 `rolled_back:canary_10`
- 游标 + 回滚计数 → 原地重试，最多 N 次

总结

主线 + 分支 + 并行 + 中断 + 循环全部就位。下一节：调试与可视化。

八、调试与可视化

1. `draw_mermaid()`：生成可读流程图（文本 + 可选 PNG）
2. `graph.stream(..., stream_mode="updates")`：逐步查看“哪个节点、返回什么增量”
3. `graph.get_state(config)`：查看任意 thread 当前快照 + 待批 interrupt
4. `langgraph dev`：挂到本地 Studio，UI 内 approve / reject

8.1 提示词 P6

代码块

```
任务：在 demos/05_loops/ 的基础上，新建 demos/06_visualize/，
**代码结构完全保留**（图就是 05 那张），只加 4 个观察 / 调试入口。

【新增 visualize.py】
- mermaid_text(graph) → graph.get_graph().draw_mermaid()
- export_mermaid(graph, out_dir)：写 .mmd 文件；try draw_mermaid_png() 写 .png，
  失败把异常塞 png_status 字段但 .mmd 一定保留
- stream_nodes(graph, initial, config)：graph.stream(..., stream_mode="updates") 遍历，
  普通 {node: update} → 追加 node；"__interrupt__" 键 → 对每个 intr.value 追加
  f"⏸ {intr.value['interrupt_id']}"。

【新增 studio_graph.py（一行包装）】
from graph import build_graph
graph = build_graph(None)        # langgraph dev 自己注入内存 checkpointer
```

```
15
16  【新增 langgraph.json】
17  { "graphs": { "release_06": "./studio_graph.py:graph" }, "dependencies":
    ["."], "env": ".env" }
18
19  【改 app.py：新增两个子命令】
20  - visualize      → print Mermaid 文本 + 报 .mmd / .png 路径
21  - watch <tid>    → 用 stream_nodes 跑一次，打印执行顺序
22
23  【验收 AC】
24  AC-1: mermaid_text 至少含 "graph TD" / "intake" / "tool_checks" / "canary_step"
    /
25        "auto_rollback" / "begin_revision" / "finalize"
26  AC-2: export_mermaid 之后 .mmd 文件存在；info 字典含 png_status 字段
27  AC-3: M 级未批工单 stream_nodes 结果前 4 个 = [intake, tool_checks,
    collect_checks,
28        classify_change]、末尾 = "⏸️ INT-1"，且列表里**没有** "approval_m"
29  AC-4: submit 后 get_state().next==("approval_m",) 且
    tasks[0].interrupts[0].value
30        ["interrupt_id"]=="INT-1"；resume approve 后下一处待批 id == "INT-3-
    canary_10"
31  AC-5: studio_graph.graph 可 import 且非空；langgraph.json 中 graphs.release_06 正
    确
32
33  【明确不做】
34  - 不修改 05 的图、节点、State，只加观察入口
35  - CI 不真起 langgraph dev（不依赖端口），AC-5 只验配置可加载
```

8.2 跑起来验证

代码块

```
1  cd demos/06_visualize
2  python -m pytest tests/ -v        # 5 条 AC
3  python app.py visualize           # 看 Mermaid 文本与 .mmd / .png 路径
4  python app.py watch 06-demo-001   # 看一次"跑到中断为止"的节点触发序列
```

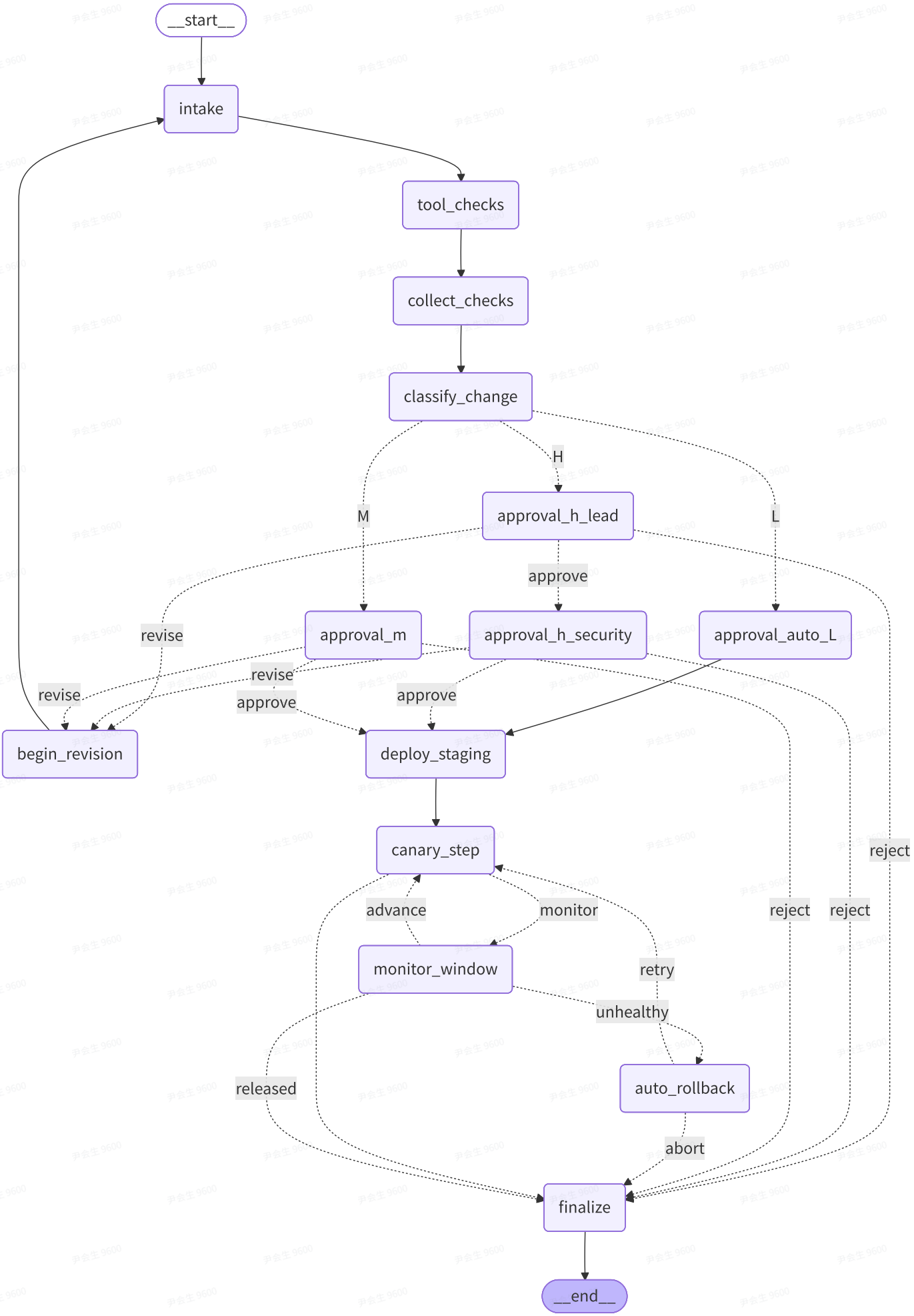
执行结果：

代码块

```
1  graph TD;
2      intake(intake)
3      tool_checks(tool_checks)
4      collect_checks(collect_checks)
```

```
    classify_change(classify_change)
    approval_m(approval_m)
    begin_revision(begin_revision)
    canary_step(canary_step)
    monitor_window(monitor_window)
    auto_rollback(auto_rollback)
    finalize(finalize)
    ...
```

视图


__start__
intake
tool_checks
collect_checks
classify_change
H
M
L
approval_h_lead
approve
approval_m
approval_h_security
approval_auto_L
revise
revise
revise
approve
approve
begin_revision
deploy_staging
canary_step
reject
advance
monitor
reject
reject
monitor_window
retry
unhealthy
released
auto_rollback
abort
finalize
__end__

`watch` 把 stream_mode 转成执行顺序：

```
代码块
1  执行顺序:  intake → tool_checks → collect_checks → classify_change → ⏸️ INT-1
```

执行：

```
代码块
1  # 06 目录下需要装 langgraph-cli[inmem]，已在顶层 requirements-dev.txt 给出
2  pip install -r ../../requirements-dev.txt
3
4  cd demos/06_visualize
5  langgraph dev              # 自动开 http://127.0.0.1:8123
```

8.3 可视化四件事

`06_visualize/visualize.py::mermaid_text`：

```
代码块
1  def mermaid_text(graph) -> str:
2      return graph.get_graph().draw_mermaid()
```

1. Mermaid 来自已编译的图

- `draw_mermaid()` 从 compile 后的图对象生成，不改源码
- 改边 / 加节点 / 调路由，图自动更新
- 条件边标签来自路由返回类型 `Literal[...]`
- `.mmd` 先落盘，PNG 在线渲染失败不丢文本

`06_visualize/visualize.py::stream_nodes`：

```
代码块
1  for update in graph.stream(initial, config=config, stream_mode="updates"):
2      for node, payload in update.items():
3          if node == "__interrupt__":
```

```
            for intr in payload:
                fired.append(f"⏸ {intr.value['interrupt_id']}")
        else:
            fired.append(node)
```

2. `stream_mode="updates"` 的 interrupt 特殊行为

- 正常事件：`{节点名：状态增量}`
- 触发 `interrupt()` 时：改发 `{"interrupt": [Interrupt(...)]}`
- 日志里“没看到 approval_m” ≠ 没执行，多半是停在 `interrupt()`
- 排查：看 `interrupt` 或 `get_state`

`06_visualize/app.py::status`（沿用 04 的 `get_state`）：

代码块

```
snap = g.get_state(cfg)
print(snap.next)                          # 待执行节点，例如 ("approval_m",)
for t in snap.tasks:                      # 每个 task 携带它的 interrupts
    for intr in (t.interrupts or []):
        print(intr.value["interrupt_id"])
print(snap.values["outcome"])             # 完整 state 字典
```

3. `get_state()` 跨进程查看状态

- 返回 `StateSnapshot`，关键字段：
  - `values`：当前完整 State
  - `next`：接下来要执行的节点
  - `tasks`：待执行任务，含 `interrupts`
- 读 SQLite，任何进程 / 时刻可查，无需图在跑
- 异步版 `aget_state()` 需 `await`

`06_visualize/langgraph.json` 与 `studio_graph.py`：

代码块

```
# studio_graph.py
from graph import build_graph
graph = build_graph(None)      # 不带 checkpointer
```

4. `langgraph dev` 注入宿主 checkpointer

- 读 `langgraph.json` → `studio_graph.py:graph`
- `build_graph(None)`：核心图不变，checkpointer 交给宿主
- 生产仍用 `app.py` + SQLite，互不影响
- 注意：dev 启用 `blockbuster`，阻塞 I/O 需用同步 `def`

九、三个实际工程问题

问题一：恢复时，审批节点从头再执行

- 现象：resume 后节点函数从头重跑，不是从 `interrupt()` 行继续
- 原因：LangGraph 只保存 State 和检查点，不保存 Python 调用栈
- 风险：`interrupt()` 前的副作用（部署、写文件、扣款）可能重复执行
- 避免：副作用放到审批通过后的独立节点；外部请求用幂等键
- 提醒：恢复会重跑代码，不会撤销已发生的外部操作

问题二：复用旧 checkpoint 污染测试

- 现象：单独跑通过，重复跑或并行跑状态异常（如 `canary_cursor` 停在 1，预期 3）
- 原因：同一 SQLite + 同一 `thread_id`，读到上一轮旧 checkpoint
- 避免：每会话用干净 checkpoint DB；不同用例用不同 `thread_id`；fixture 清理
- 提醒：测试不仅要准备输入，也要准备干净的持久化状态

问题三：异步节点里阻塞 I/O 卡住事件循环

- 现象：命令行正常，`langgraph dev` 中 `finalize` 写报告被 blockbuster 拦截
- 原因：`async def` 中执行同步文件 I/O（`os.mkdir`、`Path.write_text`），阻塞事件循环
- 避免：短同步操作写成普通 `def`（线程池执行）；必须异步则改用异步 I/O；多环境验证
- 提醒：`async def` ≠ 内部操作异步，逐项检查 I/O 是否阻塞

本章小结：把业务流程变成可执行、可恢复的图

演进路径

主线 → 条件分支 → ToolNode 并行 → 人工审批 + Checkpoint → 带预算循环 → 可视化

六条主线

1. State 存共享数据；节点干活并返回增量；边决定下一步。
2. 条件路由选路径，节点执行工作。先想清业务规则，再映射到节点和边。
3. 并行更新、列表累加要配 reducer（如 messages、审批记录）。
4. `interrupt()` 等人，Checkpoint 保存恢复；`Command(resume=...)` 送回决定；恢复时节点从头重跑。
5. 循环必须有业务上限；`recursion_limit` 只是安全阀，不能替代“最多修订几次”。
6. 跑通 ≠ 可靠：查副作用重放、测试状态隔离、异步 I/O 是否正确。

工程结构

- 六个 demo：各加一个机制
- 顶层 `release_pipeline/`：合成完整案例
- 复现：从 `demos/01_minimal/` 开始，逐步跑测试、看更新与检查点

最终要能说清

- 为什么这样分支
- 在哪里等待人工
- 状态怎样恢复
- 异常时如何安全结束

AI 辅助 vs 工程师把关

AI 可协助生成 `state.py`、节点、图装配、测试初稿

工程师必须确认

1. 明确边界：State 字段、节点、工具、明确不做什么
2. 定义 AC：覆盖业务规则、拒绝、超限等边界
3. 检查持久化测试：起点干净，旧状态不影响断言
4. 审查中断前后副作用：恢复重跑不会重复部署/发消息/提交

5. 验证实际运行形态：本地、pytest、服务/Studio 都要验

AI 把明确结构写成代码；结构是否正确、异常是否安全、结果是否符合预期，由工程师负责。